

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA06

COURSE OUTCOME COVERED

CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and

Convex-Hull problems (BL1, BL2, BL3)

Assessment Tool Weightage: 10%

QUESTIONS: 10

Total Marks:50

Annexure A

DEBUGGING & OPTIMISATION TASK

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	Sasikumar Megha	Reg. No.:	192471038
Section / Batch:	CS KBS	Date:	3/8/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

95
100

Code of Conduct

I, Sasikumar Megha (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Megha

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach, requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement, inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	/	/		
Debugging	25%	/	/		
Optimization	20%	/	/		
Analysis	20%	/	/		
Code Quality	15%				
Total Marks (100):		95			

Faculty In-charge	Course Coordinator	Head of Department
Name: <u>[Signature]</u>	Name: _____	Name: _____
Signature: <u>[Signature]</u>	Signature: _____	Signature: _____
Date: <u>2/1/24</u>	Date: _____	Date: _____

Annexure A

DEBUGGING & OPTIMISATION TASK

Q91. Debugging Selection Sort Code (Minimum Index Updated with Element Value in C)

The following Selection Sort implementation in C incorrectly stores the minimum element value in the variable meant for the minimum index, which causes invalid swapping and unexpected output for many arrays. Carefully analyze the comparison and update logic to identify why index tracking fails during each pass. Apply systematic debugging so that only positions are stored in the minimum index variable throughout the algorithm. Optimize the corrected implementation by avoiding unnecessary swaps when the current element is already the smallest. Analyze the time complexity of the corrected version and rewrite the final optimized C code with proper comments.

```
void selectionSort(int arr[], int n) {
    int i, j, min_idx, temp;
    for (i = 0; i < n; i++) {
        min_idx = i;
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = arr[j]; // ERROR
            }
        }
        temp = arr[i];

        arr[i] = arr[min_idx];
        arr[min_idx] = temp;
    }
}
```

RESULT

```
#include <stdio.h>
```

```
// Function to perform Selection Sort
void selectionSort(int arr[], int n)
{
    int i, j, min_idx, temp;

    for(i = 0; i < n - 1; i++)
    {
        // Assume current element is minimum
        min_idx = i;

        // Find index of smallest element
        for(j = i + 1; j < n; j++)
        {
            if(arr[j] < arr[min_idx])
            {
                min_idx = j; // Corrected
            }
        }

        // Swap only if needed
        if(min_idx != i)
        {
            temp = arr[i];
            arr[i] = arr[min_idx];
            arr[min_idx] = temp;
        }
    }
}

// Display array
```

```
void printArray(int arr[], int n)
{
    for(int i=0;i<n;i++)
        printf("%d ", arr[i]);
}

int main()
{
    int arr[] = {64,25,12,22,11};
    int n = sizeof(arr)/sizeof(arr[0]);

    selectionSort(arr,n);

    printf("Sorted Array: ");
    printArray(arr,n);

    return 0;
}
```

Q92. Debugging Bubble Sort Code (Incorrect Inner Loop Bound in C)

The following Bubble Sort implementation in C uses an incorrect inner loop

boundary, which causes out-of-range access or leaves some adjacent pairs unchecked depending on input size. Carefully inspect the loop condition and explain why the algorithm does not safely compare valid neighboring elements in every pass. Debug the code step-by-step so that each pass compares only valid adjacent elements within the unsorted region. Optimize the corrected version using a swap flag to terminate early when the array becomes sorted. Analyze the algorithm in best, average, and worst cases and rewrite the final C program with proper comments.

```
void bubbleSort(int arr[], int n) {  
    int i, j, temp;  
    for (i = 0; i < n; i++) {  
        for (j = 0; j <= n - i - 1; j++) { // ERROR  
            if (arr[j] > arr[j + 1]) {  
                temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
            }  
        }  
    }  
}
```

RESULT

```
#include <stdio.h>  
  
// Bubble Sort Function  
void bubbleSort(int arr[], int n)  
{  
    int i, j, temp;  
    int swapped;  
  
    for(i = 0; i < n - 1; i++)  
    {  
        swapped = 0;
```

```

        // Compare adjacent elements
        for(j = 0; j < n - i - 1; j++)
        {
            if(arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }

        // Stop if already sorted
        if(swapped == 0)
            break;
    }
}

void printArray(int arr[], int n)
{
    for(int i=0;i<n;i++)
        printf("%d ", arr[i]);
}

int main()
{
    int arr[] = {5,1,4,2,8};
    int n = sizeof(arr)/sizeof(arr[0]);

    bubbleSort(arr,n);

```



```
printf("Sorted Array: ");  
printArray(arr,n);  
  
return 0;  
}
```

The following Linear Search implementation in C returns failure from inside the loop immediately after the first non-matching comparison, which prevents the rest of the array from being searched. Carefully analyze the control flow and explain why valid matches later in the array are never reached. Apply systematic debugging so that the search continues until the key is found or the full array has been checked. Optimize the corrected implementation for clarity and direct readability. Analyze the time complexity of the corrected solution and rewrite the final C code with suitable comments.

```
int linearSearch(int arr[], int n, int key) {
    int i;
    for (i = 0; i < n; i++) {
        if (arr[i] == key)
            return i;
        else
            return -1; // ERROR
    }
    return -1;
}
```

RESULT

```
#include <stdio.h>
```

```
// Linear Search Function
```

```
int linearSearch(int arr[], int n, int key)
{
    int i;

    for(i = 0; i < n; i++)
    {
        if(arr[i] == key)
            return i;
    }
}
```

```
// Key not found
return -1;
}

int main()
{
    int arr[] = {10,20,30,40,50};
    int n = sizeof(arr)/sizeof(arr[0]);
    int key = 40;

    int result = linearSearch(arr,n,key);

    if(result != -1)
        printf("Element found at index %d", result);
    else
        printf("Element not found");

    return 0;
}
```

The following Binary Search implementation in C updates the lower and upper

bounds in the wrong direction, making it discard the valid half of the sorted array during search. Carefully inspect the comparison logic and explain why the search interval moves away from the target instead of toward it. Debug the code step-by-step so that the midpoint decisions correctly preserve the remaining candidate region. Optimize the corrected implementation for clarity and proper edge-case handling. Analyze the best, average, and worst-case complexity and rewrite the final C code with proper documentation.

```
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1, mid;
    while (low &lt;= high) {
        mid = (low + high) / 2;
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] &lt; key)
            high = mid - 1; // ERROR
        else
            low = mid + 1; // ERROR
    }
    return -1;
}
```

RESULT

```
#include <stdio.h>
```

```
// Binary Search Function
```

```
int binarySearch(int arr[], int n, int key)
{
    int low = 0;
    int high = n - 1;
```

```

while(low <= high)
{
    int mid = low + (high - low) / 2;

    if(arr[mid] == key)
        return mid;

    else if(arr[mid] < key)
        low = mid + 1;    // Corrected

    else
        high = mid - 1;    // Corrected
}

return -1;
}

int main()
{
    int arr[] = {10,20,30,40,50,60,70};
    int n = sizeof(arr)/sizeof(arr[0]);
    int key = 50;
    int result = binarySearch(arr,n,key);
    if(result != -1)
        printf("Element found at index %d", result);
    else
        printf("Element not found");

    return 0;
}

```

Q95. Debugging Closest Pair Code (Wrong Initialization and Comparison in C)

The following brute force Closest Pair implementation in C initializes the minimum

distance incorrectly and uses the wrong comparison direction, leading to incorrect results even for simple point sets. Carefully analyze why the algorithm never preserves the actual smallest distance among all pairs. Apply systematic debugging to correctly initialize the minimum distance and update it only when a smaller valid distance is found. Optimize the implementation by avoiding redundant distance calculations where possible. Analyze the time complexity of the corrected solution and rewrite the final C code with proper comments.

```
#include <math.h>

double closestPair(int x[], int y[], int n) {
    int i, j;
    double minDist = 0; // ERROR
    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {
            double dist = sqrt((x[i] - x[j]) * (x[i] - x[j]) +
                               (y[i] - y[j]) * (y[i] - y[j]));
            if (dist > minDist) { // ERROR
                minDist = dist;
            }
        }
    }
    return minDist;
}
```

RESULT

```
#include <stdio.h>
#include <math.h>

// Function to find closest pair distance
double closestPair(int x[], int y[], int n)
{
    int i, j;
```

```

// Initialize with first pair distance
double minDist = sqrt((x[0]-x[1])*(x[0]-x[1]) +
                      (y[0]-y[1])*(y[0]-y[1]));

for(i = 0; i < n - 1; i++)
{
    for(j = i + 1; j < n; j++)
    {
        double dx = x[i] - x[j];
        double dy = y[i] - y[j];
        double dist = sqrt(dx*dx + dy*dy);

        // Update minimum distance
        if(dist < minDist)
            minDist = dist;
    }
}

return minDist;
}

int main()
{
    int x[] = {2,12,5,9};
    int y[] = {3,30,1,7};
    int n = 4;

    printf("Minimum Distance = %.2lf\n", closestPair(x,y,n));

    return 0;
}

```